10 — Mixture of Experts

How a model can hold 47 billion parameters but use only 13 billion of them for each token, and what that trade costs in training stability, network traffic and decoding speed.

Reading time: about 50 minutes. You need: modules 00 to 05. Two ideas from them matter here, and both are restated where they are used: collective operations (module 04) and arithmetic intensity (module 05). You do not need: a GPU, or any experience of distributed training.

Three different sizes for the same model

Mixtral 8x7B is a well-documented open-weights model. Its name suggests eight copies of a 7-billion parameter model, so 56 billion in total. The weight files hold 46.7 billion. The compute cost of one token matches a model of 12.9 billion.

Three numbers, all correct, all describing the same thing. None of them is a rounding of another. The gap between the second and the third is the whole subject of this article.

A dense model has one size. Every parameter is read and multiplied for every token, so "how big is it" has one answer. A **mixture of experts**, or MoE, breaks that link. It stores many parameters and uses a few of them per token, chosen by a small network called a router.

That buys capacity without buying arithmetic. It also introduces a new way for training to fail, a new communication pattern, and a decoding penalty that gets worse as you serve more users. This article works through all four.

What you will learn

By the end of this article, you will be able to:

- count the total and per-token-active parameters of an MoE model, and say which one matters when;
- implement top-k routing, and explain why the gate value multiplies the expert's output;
- describe how a router collapses onto one expert, and fix it with an auxiliary loss;
- explain capacity factors, token dropping, and expert-choice routing;
- say why an MoE is cheap to train, awkward to shard, and expensive to decode from.

Section 1 counts parameters. Sections 2 to 4 cover routing and the ways it goes wrong. Sections 5 and 6 cover the two systems consequences. A recap and the exercise briefing follow at the end.

---

1. Parameters are cheap, arithmetic is not

A transformer block has two parts: attention, which mixes information between tokens, and a feed-forward network, which processes each token on its own. Module 02 section 4 covers the second part. In a large model it holds most of the parameters.

An MoE replaces that one feed-forward network with E copies of it, called **experts**, plus a small linear layer called the **router**. Each token is sent to k of the E experts, with k much smaller than E. Attention is left alone.

The arithmetic follows immediately. Storing the layer costs E experts. Running one token through it costs k experts. The ratio E/k is how much extra capacity you store per unit of arithmetic you pay.

Counting a real model

Mixtral 8x7B has 32 layers, hidden size $h = 4096$, and a feed-forward hidden size of 14,336. It uses $E = 8$ experts with $k = 2$, and grouped-query attention with 32 query heads and 8 key-value heads. Each expert has three matrices, because the activation function is SwiGLU.

Piece	Formula	Parameters
attention, per layer	$2h^2 + 2h\,n_{kv}d_{head}$	41.94 M
one expert	$3h \times 14{,}336$	176.16 M
router, per layer	$h \times E$	0.03 M
total	32(attn + 8 exp) + embeddings	**46.70 B**
active per token	32(attn + 2 exp) + embeddings	**12.88 B**

Take the expert row. One expert holds three matrices of shape $4096 \times 14{,}336$, which is $3 \times 4096 \times 14{,}336 = 176{,}160{,}768$ parameters. Eight of those is 1.409 billion per layer, against 41.94 million for that layer's attention. The experts are 97% of the model.

The published figures for Mixtral are 46.7 B and 12.9 B. Counting the matrices one by one reproduces both, which is worth doing once so that the numbers stop being quoted facts.

What the gap is actually worth

A ratio of $46.70/12.88 = 3.63$ looks like free capacity. It is not free, and the honest comparison is against a dense model of the same shape. Set $E = 1$ in the same configuration and you get a 7.24-billion parameter model.

So Mixtral performs 1.78 times the arithmetic of that dense model, and occupies 6.45 times its memory. In 16-bit weights that is 93.4 GB against 14.5 GB. The trade is real, but it is memory for quality at fixed speed, not capacity for nothing.

Which number matters depends on what is scarce. During training, compute is usually the binding constraint, and the active count is what sets the step time. During serving, memory and bandwidth are usually binding, and the total is what you must fit and read.

> **In the interview:** if asked "how big is Mixtral", give all three numbers and say what each one governs. Answering "47 billion" alone suggests you have not thought about where the cost lands.

---

2. The router: how a token picks its experts

The router is a single linear layer, from the hidden size to the number of experts. It produces one score per expert for each token. Those scores go through a softmax, and the largest k are kept.

Take one token, four experts and $k = 2$. Suppose the router produces these scores:

```
logits      +2.1   -0.4   +1.3   +0.7
softmax    0.562  0.046  0.253  0.139
top-2      expert 0 (0.562) and expert 2 (0.253)
gates      0.690  and  0.310        after renormalising over the two kept
```

The softmax step is the one from module 00. Exponentiating the logits gives 8.166, 0.670, 3.669 and 2.014, which sum to 14.520, and $8.166/14.520 = 0.562$. The renormalising step then divides each kept probability by their sum: $0.562/(0.562 + 0.253) = 0.690$.

The token is passed through experts 0 and 2 only. Their outputs are combined as a weighted sum, using 0.690 and 0.310 as the weights. Experts 1 and 3 are not evaluated for this token at all, and that is where the saving comes from.

Renormalising is a choice, not a requirement. It makes the weights sum to one, so a layer whose experts are all identical behaves exactly like a single dense expert. The exercise checks that property, and it holds to 6×10^{-8}.

Why the gate value multiplies the output

This detail looks cosmetic and is not. The router's output is used twice: once to decide *which* experts run, and once as a multiplier on *what they return*.

The deciding step uses `topk`, which returns an index. An index is a discrete choice, and discrete choices have no derivative. If the gate value did not also multiply the output, the router's weights would never appear in the loss as a continuous quantity, and gradient descent could not train them.

This is measurable. Run one backward pass through an MoE layer, then inspect the router's gradient:

```
router gradient with the gate multiply   : 170.4598
router gradient without it               : None
```

`None`, not zero. Without the multiply, PyTorch does not record the router in the computation graph at all, because nothing differentiable depends on it. The router would keep its random initialisation for the whole of training.

So the gate has two jobs. It weights the experts' contributions, and it is the only channel through which the router learns anything. Keep that in mind for section 3, because it also explains how the router learns the wrong thing.

> **In the interview:** "how does the router get a gradient if top-k is not differentiable" is a standard question. The answer is that the chosen gate multiplies the output, so the loss is a smooth function of the router's weights even though the selection is not.

---

3. Why routers collapse

Here is the failure that makes MoE training harder than it looks. Nothing in the loss asks the router to use all the experts. It asks for low loss, and one expert that works is enough.

The feedback loop

Suppose expert 3 is slightly better than the others early in training, for any reason at all. Then the router raises its score, so expert 3 receives more tokens. More tokens means more gradient updates, so expert 3 improves faster than its neighbours. Being better, it attracts more tokens still.

The loop is self-reinforcing, and the end state is one expert handling almost everything. This is called **routing collapse**. The model now has the memory cost of E experts and the capacity of one.

A common mistake is to assume this is a rare pathology that shows up only at scale. It is the default behaviour, and it appears in a model small enough to train on a laptop in twenty seconds.

What it looks like

The exercise trains an eight-expert layer with top-1 routing for 1,500 steps. One expert is given a small head start, by adding a constant to its router score at initialisation. Everything else is identical between the two runs.

```
alpha = 0.0
  final share %   :  96.3   0.5   1.3   0.7   0.1   0.0   0.5   0.6
  experts alive   : 2/8     busiest 96.3%
  busiest at step : t0:94%  t300:96%  t600:96%  t900:97%  t1200:96%

alpha = 0.01
  final share %   :  12.8  13.7  13.0  12.2  11.3  12.7  12.6  11.5
  experts alive   : 8/8     busiest 13.7%
  busiest at step : t0:94%  t300:22%  t600:21%  t900:16%  t1200:14%
```

A balanced layer would give each expert 12.5% of the tokens. The first run holds 96.3% on one expert and never recovers. The second run uses the auxiliary loss from section 4, and lands between 11.3% and 13.7% across all eight.

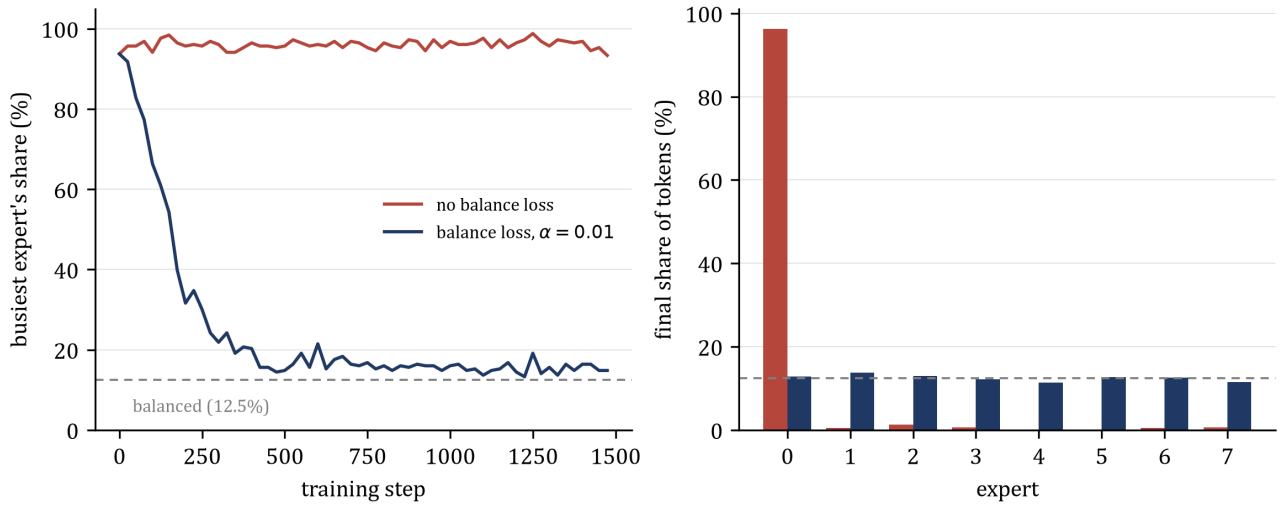


Figure 1: Expert load during training. Left: the share held by the busiest expert at each step. Right: the final share of each expert. The collapsed run starts at 94% and stays there; the balanced run starts in the same place and is back near 12.5% within 500 steps.

The second run is the more interesting one. It begins as collapsed as the first, at 94%, and is pulled apart during training. The balance pressure does not merely prevent collapse; it can reverse one that has already happened.

---

4. Four ways to keep the experts busy

The auxiliary load-balance loss

The standard fix adds a term to the loss that is minimised when the load is even. Write f_i for the fraction of tokens routed to expert i, and P_i for the mean router probability assigned to expert i across the batch. The Switch Transformer auxiliary loss is

$$\mathcal{L}_{\text{aux}} = \alpha E \sum_{i=1}^{E} f_i P_i.$$

In words: multiply how busy each expert was by how much the router wants to use it, add those up over the experts, and scale. The factor E makes a perfectly balanced layer score 1 for any number of experts, so α means the same thing as E changes.

The product of two quantities looks arbitrary until you ask which one carries the gradient. The load f_i comes from counting tokens, and counting is not differentiable, so f_i is a constant as far as backpropagation is concerned. All the gradient flows through P_i.

That makes the loss readable as a weighted penalty on router probability. Experts with a large f_i get a large weight on their P_i, so the gradient pushes probability away from whichever experts are already busy. An expert nobody used has $f_i = 0$ and is not penalised at all, which is why probability flows back to it.

The value of α is a real trade-off. Too small and the loss does nothing; too large and the router is pushed towards uniform routing, which throws away the specialisation that motivated the architecture. Switch Transformer uses $\alpha = 0.01$, which is the value used above.

Capacity, and dropping tokens

The auxiliary loss encourages balance but does not guarantee it. A real implementation still has to decide how much memory to give each expert before the batch arrives, because buffers must be allocated in advance.

That allocation is the **capacity**: a fixed number of tokens each expert will accept. It is set as a multiple of the even share, and the multiplier is the **capacity factor**.

$$\text{capacity} = \text{capacity factor} \times \frac{Tk}{E}$$

with T tokens in the batch. Tokens arriving at an expert that is already full are **dropped**: not computed at all, and passed through the residual connection unchanged. A dropped token still produces an output, through the skip path, so nothing crashes. It is quietly a worse output.

Here is a mildly skewed router, with its busiest expert at 23% against an even share of 12.5%:

Capacity factor	Tokens dropped
1.00	12.3%
1.25	7.6%
1.50	4.5%
2.00	0.0%

Reading the first row: at capacity factor 1.00 each expert accepts $1.00 \times 512 \times 2/8 = 128$ token-expert pairs. The batch holds 1,024 pairs in all, and 12.3% of them arrive at an expert that has already taken its 128. Doubling the buffer removes the problem entirely, at twice the memory and twice the padding.

Perfectly balanced routing drops nothing at capacity factor 1.00, which the exercise also checks. The dropping is caused by imbalance, not by the mechanism.

Expert choice: let the experts pick

The three mechanisms above all patch the same underlying problem: tokens choose experts, so nothing stops them from crowding. **Expert-choice routing** inverts the selection. Each expert takes its own top-c tokens, where c is its capacity.

Balance then holds by construction. Every expert processes exactly c tokens, no auxiliary loss is needed, and no token is dropped for lack of room. The guarantee moves from "encouraged by a penalty term" to "true by the shape of the operation".

The price is that the per-token guarantee disappears. With 512 tokens, 8 experts and capacity 128, the exercise measures:

```
tokens served 0 times: 77, once: 147, more: 288
```

Seventy-seven tokens got no expert at all, and pass through on the residual alone. This is a genuine trade, not a free win. It also breaks causal decoding, because an expert cannot rank a token against tokens that have not been generated yet. Expert choice is used during training, not during autoregressive inference.

The router z-loss

One more term appears in most production recipes. The router's logits can grow large, which makes the softmax numerically delicate in low precision and the routing decisions brittle. The **z-loss** penalises the log of the softmax denominator, keeping the logits small.

$$\mathcal{L}_z = \frac{1}{T}\sum_{t=1}^{T}\Big(\log\sum_{i=1}^{E} e^{\ell_{ti}}\Big)^2$$

Here ℓ_{ti} is the router logit for token t and expert i. The ST-MoE paper introduced it after finding that MoE training diverged in bfloat16 where dense training did not. It is cheap insurance and is now standard.

---

5. Expert parallelism: the all-to-all

Module 04 covered three ways to split a model across devices: by data, by tensor, and by layer. An MoE offers a fourth, and it is the most natural split in the course. The experts are already independent functions, so you put different experts on different devices.

This is **expert parallelism**. With 8 experts and 4 devices, each device holds 2 experts. No matrix is cut in half and no layer boundary is crossed, which makes the partition cleaner than tensor or pipeline parallelism.

What has to move

The difficulty is that tokens do not arrive where their expert lives. A device holds whichever tokens the batch gave it, and those tokens were routed by content, so they want experts scattered across every device.

So each MoE layer has two communication steps:

1. **Dispatch.** Every device sends each of its tokens to the device owning the expert it chose.
2. **Combine.** Every device sends each result back to the device the token came from.

Both are an **all-to-all**: every device sends a different piece of data to every other device. Module 04 lists it among the five collectives. It is the one that appears nowhere else in this course, and it is the awkward one.

An all-reduce moves the same data to everyone and can be built from a ring, so each device sends to one neighbour at a time. An all-to-all cannot. Every pair of devices exchanges something different, so the pattern needs the full bisection bandwidth of the interconnect rather than a ring of neighbour links. It is also a synchronisation point: nothing proceeds until the slowest device has sent everything.

What it costs

Each token-expert pair sends one hidden vector out and one back. With T tokens, k experts each, hidden size h and 2-byte values, one hop moves $Tkh \times 2$ bytes.

Tokens per step	Token-expert pairs	One hop	Per layer	Whole model, 32 layers
1,024	2,048	16.8 MB	33.6 MB	1.07 GB
4,096	8,192	67.1 MB	134.2 MB	4.29 GB
16,384	32,768	268.4 MB	536.9 MB	17.18 GB

Take the middle row. 4,096 tokens at $k = 2$ give 8,192 token-expert pairs, each carrying a vector of 4,096 numbers at 2 bytes, so $8{,}192 \times 4{,}096 \times 2 = 67.1$ MB per hop. Dispatch and combine make 134.2 MB per layer, and 32 layers make 4.29 GB for one forward pass.

Two practical consequences follow. Expert parallelism is kept inside one machine wherever possible, for the same reason module 04 gives for tensor parallelism: the links inside a node are far faster than the links between nodes. And the capacity factor from section 4 is what makes these buffers a fixed size, known before the batch is routed, which is what lets the collective be issued at all.

The part that is easy to check

None of this changes the arithmetic. The same tokens meet the same experts, whatever device each expert sits on. So a sharded implementation must produce bit-comparable gradients, and that is testable on one machine without any distribution at all:

```
1 device(s): 512 token-expert pairs move, output identical True, gradients identical True
2 device(s): 512 token-expert pairs move, output identical True, gradients identical True
4 device(s): 512 token-expert pairs move, output identical True, gradients identical True
8 device(s): 512 token-expert pairs move, output identical True, gradients identical True
```

This is the same discipline as exercise 04. A sharded computation either reproduces the unsharded gradients or it has a bug, and that question does not need a cluster to answer.

---

6. Why decoding gets worse, not better

Module 05 established that generating a token is limited by memory bandwidth, not arithmetic. The model must read every weight it uses to produce one token, and reading is slower than multiplying. The measure it used is **arithmetic intensity**: floating-point operations performed per byte read from memory. Higher is better, and batching raises it, because a batch shares one reading of the weights across many tokens.

That last clause is where MoE breaks the pattern. A batch shares one reading of the weights only if its tokens use the same weights.

How many experts does a batch touch?

Take a balanced router, E experts, top-k. One token reaches k experts. A second token picks its own k, probably overlapping. As the batch grows, the union grows, and it stops growing when it has covered everything.

A given expert is missed by one token with probability $1 - k/E$, and missed by all B tokens with that raised to the B. So the expected number of distinct experts reached is

$$\mathbb{E}[\text{experts touched}] = E\left(1 - \left(1 - \frac{k}{E}\right)^{B}\right).$$

With $E = 8$ and $k = 2$, a batch of 16 already reaches 7.92 of the 8 experts.

The gap, and where it stops

Now compare against a dense model with the same active parameter count, which reads exactly k experts' worth of weights for every token. Both do the same arithmetic. They differ in what they read.

Batch	Experts touched	MoE intensity	Dense intensity	Gap
1	2.00	1.0	1.0	1.0x
4	5.47	1.5	4.0	2.7x
16	7.92	4.0	16.0	4.0x
64	8.00	16.0	64.0	4.0x
256	8.00	64.0	256.0	4.0x

Reading the batch-16 row: the batch does $16 \times 2 = 32$ experts' worth of arithmetic and reads 7.92 experts' worth of weights, giving $32/7.92 = 4.0$. The dense model does the same 32 and reads 2, giving 16. The intensity gap is 4.0 times.

The gap stops at exactly E/k, and the reason is visible in the table. Once every expert is touched, the bytes read stop growing while the arithmetic keeps growing, so both intensities rise together and their ratio freezes. With $E = 8$ and $k = 2$ that ratio is 4.

So an MoE decodes with four times less arithmetic per byte read than the dense model it matches in compute. Batching still helps it, and helps it by the same factor it helps a dense model. It starts from four times further back.

Prefill is the opposite case. A 1,000-token prompt is processed in one pass, touches every expert regardless, and has enough arithmetic to be compute-limited. There the MoE's lower active count is a straight win.

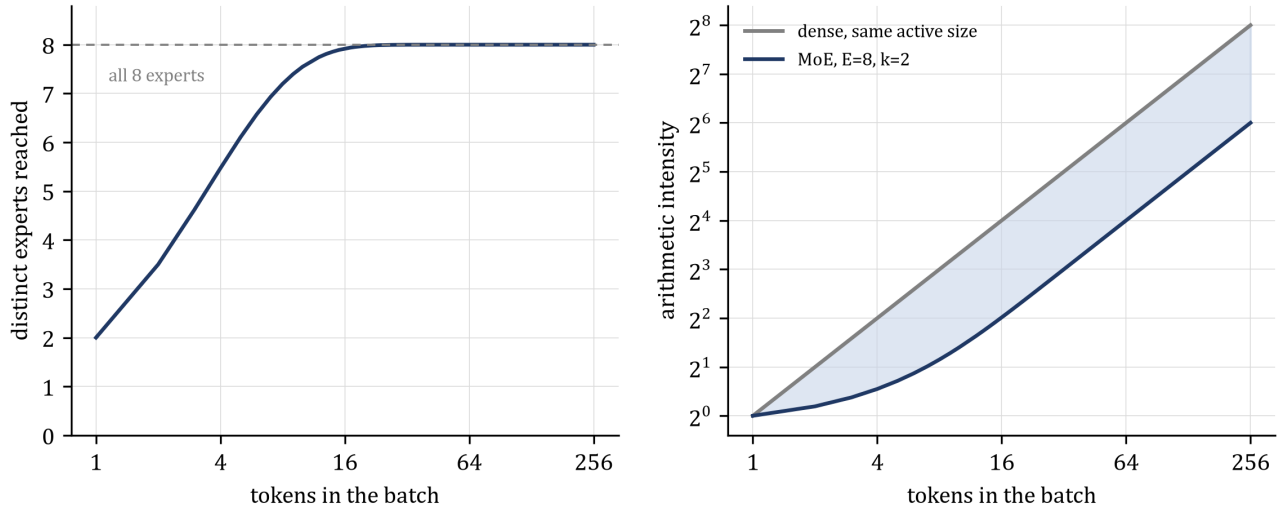


Figure 2: Left: the number of distinct experts a batch reaches, which saturates at 8. Right: arithmetic intensity against batch size, for an MoE and for a dense model with the same active parameter count. The two lines stay parallel on log axes, which means a constant ratio.

> **In the interview:** "is MoE faster at inference" has no single answer. It is faster per token than a dense model of the same *total* size, and slower per byte than a dense model of the same *active* size. Ask which baseline is meant before answering.

---

7. What breaks

Four problems recur in practice, and all of them are consequences of what the earlier sections described.

Memory, not compute, becomes the limit. The whole model must be resident even though a fraction of it runs. Section 1's 93.4 GB against 14.5 GB is the version of this you can compute in advance; it is the reason MoE serving needs more devices than its active size suggests.

Fine-tuning overfits more readily than dense fine-tuning. The model has many parameters and a small downstream dataset, and the router can specialise its experts to the fine-tuning set. A common remedy is to freeze the router, or to fine-tune only the attention layers and leave the experts alone.

Training is less stable. The router's logits can grow, the softmax can saturate, and routing decisions can oscillate between steps. The z-loss from section 4 exists for this reason.

Load imbalance costs wall-clock time even when it costs no accuracy. Devices in an expert-parallel group wait at the all-to-all for the slowest one. An expert with twice its share of tokens makes every other device idle, so balance is a throughput property, not only a quality one.

Two further design choices are worth knowing, because recent models use both. **Fine-grained experts** make each expert smaller and raise both E and k, which gives the router more combinations to choose from at the same active cost. **Shared experts** keep one or two experts that every token always visits, so common computation is not duplicated across specialists. DeepSeek-MoE uses both together.

Recap

- An MoE replaces the feed-forward network with E experts and a router, and sends each token to k of them. Attention is unchanged.
- Total parameters scale with E, active parameters with k. Mixtral is 46.70 B total and 12.88 B active, but costs 6.45 times the memory of the dense model of its own shape.
- The gate value must multiply the expert's output, because `topk` has no gradient and the multiply is the router's only path to the loss.
- Routers collapse by positive feedback. Without a balance loss, one expert took 96.3% of tokens; with $\alpha = 0.01$, the busiest held 13.7%.
- Capacity factors bound the buffers and drop the overflow. Expert choice makes balance exact, at the cost of leaving some tokens unserved, and cannot be used for causal decoding.
- Expert parallelism needs two all-to-alls per layer, which is the hardest collective to scale across machines.
- Decoding intensity is worse than a dense model of equal active size by a factor that saturates at E/k.

The exercise: route, collapse, balance, shard

What you will build. Six pieces, each checked numerically:

1. a parameter counter that reproduces Mixtral's published 46.7 B and 12.9 B;
2. a working MoE layer, verified against the dense MLP it reduces to;
3. a reproduction of routing collapse, and the auxiliary loss that reverses it;
4. capacity-based token dropping, and expert-choice routing;
5. expert parallelism in one process, checked for identical gradients;
6. the intensity calculation that explains the decoding penalty.

Time and setup. Allow two to three hours. The setup is the same as for module 00, and everything runs on a CPU. The reference solution takes 31 seconds end to end, almost all of it in part 3, which is the only part that trains anything.

How to run it. Open `10-mixture-of-experts/exercise.ipynb`, or run `python exercise.py` from inside the module folder.

How the checks work. They work as in module 00. Each check prints `[ok]` or `[FAIL]` and keeps going, and `report()` summarises a part. Before you start, every part reports that it is not implemented, one after another:

```
[FAIL] 1  -- NotImplementedError:

0/1 passed.  Still failing: 1
[FAIL] 2a  -- NotImplementedError:

0/1 passed.  Still failing: 2a
```

Step map. This table links each part to the section of the article it practises.

Part	You write	The idea it practises	Section
1	`moe_params`	total against active	1
2	`MoELayer.forward`, `load_balance_loss`	routing, and the gate's gradient	2

Part	You write	The idea it practises	Section
3	`train_moe`	collapse and the balance loss	3, 4
4	`apply_capacity`, `expert_choice`	dropping, and inverted routing	4
5	`moe_expert_parallel`	the all-to-all, and what it preserves	5
6	`experts_touched`, `moe_intensity`	the decoding penalty	6

One trap in part 2 deserves a warning. A token can be routed to the same expert in only one slot, but several tokens land on one expert at once, so you cannot write results with plain indexing without losing some. Use `index_add_`, and the gates will accumulate correctly.

When you are done. All 26 checks pass. Your collapsed run puts over 90% of tokens on one expert, your balanced run keeps the busiest below 50%, and your sharded MoE reproduces the unsharded gradients for 1, 2, 4 and 8 simulated devices.

Several parts end with short written questions, such as why dropped tokens do not crash the model. Answer each in a sentence or two.

If you get stuck.

- For part 2, build the layer with two experts and four tokens, and print the index tensor before writing the dispatch loop.
- For part 3, check that your head start actually biases every token the same way. A perturbation of the router's weight matrix rotates it instead, and will not collapse.
- For part 5, compare outputs before gradients. An output mismatch is an indexing bug; a gradient mismatch with matching outputs is an accumulation bug.
- If you are still stuck, `solution.py` has a complete reference.

Want more? The notebook ends with four stretch goals: a shared-expert variant, the z-loss, a measurement of how much the balance loss costs in task loss, and DeepSeek-V3's auxiliary-loss-free balancing by router bias.

Going deeper

Start here

- Fedus, Zoph & Shazeer (2022), *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, JMLR 23. arXiv:2101.03961. The clearest single paper on the whole subject. Section 2 gives the auxiliary loss and the capacity factor exactly as used here.

The idea and its history

- Shazeer et al. (2017), *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, ICLR. arXiv:1701.06538. The paper that made conditional computation work at scale. Read section 4 on load balancing.
- Jacobs et al. (1991), *Adaptive Mixtures of Local Experts*, Neural Computation 3(1). The original, thirty years earlier, with the same gating idea on a tiny scale.

Balancing and routing

- Lepikhin et al. (2021), *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*, ICLR. arXiv:2006.16668. Where capacity factors and the expert-parallel all-to-all were

worked out.

- Zoph et al. (2022), *ST-MoE: Designing Stable and Transferable Sparse Expert Models*. arXiv:2202.08906. The z-loss, the fine-tuning instability, and the most honest account of what goes wrong.
- Zhou et al. (2022), *Mixture-of-Experts with Expert Choice Routing*, NeurIPS. arXiv:2202.09368. Section 3 is the inverted routing from section 4 of this article.

Models and systems

- Jiang et al. (2024), *Mixtral of Experts*. arXiv:2401.04088. The model counted in section 1.
- Dai et al. (2024), *DeepSeekMoE: Towards Ultimate Expert Specialization*. arXiv:2401.06066. Fine-grained and shared experts, with ablations for both.
- DeepSeek-AI (2024), *DeepSeek-V3 Technical Report*. arXiv:2412.19437. Section 2.1.2 replaces the auxiliary loss with a bias term adjusted during training.
- Gale et al. (2023), *MegaBlocks: Efficient Sparse Training with Mixture-of-Experts*, MLSys. arXiv:2211.15841. Block-sparse kernels that remove token dropping entirely.

Scaling behaviour

- Clark et al. (2022), *Unified Scaling Laws for Routed Language Models*, ICML. arXiv:2202.01169. How the benefit of more experts changes with model size, and where it stops.

Previous: module 09 built attention as a CUDA kernel. This module is the last, and it is the one place in the course where the feed-forward network, rather than attention, is the interesting part.